

Patrones de diseño de software

Análisis y diseño orientado a objetos del módulo de gestión de reservas de restaurantes

Brindis — Sistema de reservas de restaurantes por ocasión

Yslen Natalia Moreno Gutierrez
Jorge Antonio García Romero
Karla Mariana Duque Callejas

Universidad Sergio Arboleda
Escuela de ciencias exactas e ingeniería
Bogotá D.C., 8 de octubre de 2026

Índice

1. Introducción **5**

2. Planteamiento del problema **5**

3. Objetivos **6**

3.1. Objetivo general 6

3.2. Objetivos específicos 6

4. Alcance y propuesta de valor **6**

4.1. Alcance 6

4.2. Propuesta de valor 7

5. Análisis **8**

5.1. Actores y roles 8

5.2. Requerimientos funcionales 8

5.3. Requerimientos no funcionales 10

5.4. Reglas de negocio 10

5.5. Diagrama de casos de uso 12

5.6. Descripción de los casos de uso principales 12

5.7. Ciclo de vida de una reserva 14

6. Diseño **14**

6.1. Arquitectura MVC por capas 14

6.2. Diagrama de clases del modelo 15

6.3. Modelo relacional de la base de datos 16

6.4. Diagrama de clases DAO 17

6.5. Patrones de diseño aplicados 18

6.5.1. Patrón creacional Factory Method: creación de usuarios por rol 19

6.5.2. Patrón creacional Builder: construcción validada de la reserva 19

6.5.3. Patrón estructural Proxy: control de acceso por rol 20

6.5.4. Patrón estructural Decorator: costo de los servicios extra . . . 20

6.6. Diagrama de clases de control 21

6.7. Diagrama de secuencia: confirmar una reserva 22

6.8. Diseño de la interfaz de usuario 22

7. Implementación **23**

8. Casos de prueba **23**

9. Video explicativo **24**

10. Conclusiones **24**

Referencias **25**

Índice de tablas

1. Elementos diferenciadores de Brindis 7
2. Actores del sistema 8
3. Requerimientos funcionales 8
4. Requerimientos no funcionales 10
5. Reglas de negocio 10
6. Especificación del caso de uso CU01 Iniciar sesión 12
7. Especificación del caso de uso CU04 Registrar reserva 13
8. Especificación del caso de uso CU09 Modificar o cancelar reserva (cancelación) 13
9. Especificación del caso de uso CU10 Gestionar agenda del día 14
10. Valores de la enumeración Ocasion 16
11. Patrones de diseño aplicados en Brindis 18
12. Casos de prueba funcionales 23

1. Introducción

La reserva de mesas es uno de los puntos de contacto más importantes entre un restaurante y sus clientes. Sin embargo, la mayoría de los sistemas de reserva tratan todas las visitas de la misma manera: piden una fecha, una hora y un número de personas, y asignan la primera mesa libre. Una cena de aniversario, una propuesta de matrimonio y un almuerzo casual reciben el mismo trato, aunque cada una exige una mesa, una ambientación y un servicio diferentes.

Este documento presenta el análisis, el diseño y el desarrollo orientado a objetos de **Brindis**, un módulo web de gestión de reservas cuyo elemento diferenciador es la **reserva por ocasión**. El cliente inicia la reserva indicando qué celebra; a partir de esa elección, el sistema le sugiere una zona del restaurante, le muestra un plano interactivo para elegir una o varias mesas, le ofrece los planes de decoración que corresponden a su ocasión y le permite preordenar los platos, contratar una barra de cócteles y definir cómo pagará. El personal del restaurante, por su parte, recibe en su agenda cada reserva con las restricciones alimentarias y las indicaciones de servicio que debe preparar.

El sistema contempla tres roles (cliente, empleado y administrador), una clasificación de clientes en VIP y generales con un sistema de *strikes*, una política de cancelación con reembolsos y la emisión de comprobantes de pago. Se construye con una arquitectura Modelo–Vista–Controlador (MVC) organizada en capas, una capa de acceso a datos basada en el patrón *Data Access Object* (DAO) (Alur et al., 2003) y exactamente cuatro patrones de diseño del catálogo de Gamma et al. (1994): dos creacionales (*Factory Method* y *Builder*) y dos estructurales (*Proxy* y *Decorator*). Cada patrón resuelve un problema concreto del dominio.

2. Planteamiento del problema

Los restaurantes pequeños y medianos suelen gestionar sus reservas por teléfono, por mensajería o en cuadernos, lo que genera cinco dificultades recurrentes:

1. **Asignación ineficiente de mesas y reservas duplicadas.** Sin una vista del salón se asignan mesas inadecuadas, y cuando un empleado reserva por teléfono mientras un cliente reserva en línea, ambos pueden tomar la misma mesa.

2. **Pérdida de información del cliente.** Las alergias, el motivo de la visita y los servicios especiales se comunican de palabra y no siempre llegan a cocina ni al personal de sala.

3. **Oportunidades de venta desaprovechadas.** No se ofrecen de forma sistemática servicios adicionales (decoración, barra de cócteles, preorden del menú) que el cliente estaría dispuesto a pagar en una ocasión especial.

4. **Cancelaciones e inasistencias sin control.** Cuando un cliente cancela tarde o no se presenta, el personal ya preparó la zona y los platos, pero no existen reglas claras de reembolso ni un registro del comportamiento del cliente.

5. **Falta de control de acceso.** No se distingue qué puede hacer un cliente, un empleado o un administrador, lo que expone información de otros clientes y operaciones sensibles.

Se requiere entonces un módulo que permita al cliente reservar de forma sencilla y personalizada, y al personal controlar sus mesas, horarios, reservas y pagos con toda la información necesaria para preparar el servicio.

3. Objetivos

3.1. Objetivo general

Analizar, diseñar y desarrollar con el paradigma orientado a objetos un módulo web de gestión de reservas de restaurantes, con inicio de sesión y formulario de reservas para los roles de cliente y administrador, aplicando una arquitectura MVC por capas, dos patrones de diseño creacionales y dos estructurales.

3.2. Objetivos específicos

1. Identificar los actores, los requerimientos funcionales y no funcionales, las reglas de negocio y los casos de uso del módulo.

2. Diseñar el modelo relacional de la base de datos y su capa de acceso a datos mediante el patrón DAO.

3. Aplicar los patrones creacionales *Factory Method* y *Builder* para crear los usuarios según su rol y construir reservas válidas paso a paso.

4. Aplicar los patrones estructurales *Proxy* y *Decorator* para controlar el acceso según el rol y calcular dinámicamente el costo de los servicios extra.

5. Implementar una interfaz intuitiva que guíe la reserva en cuatro pasos, con un plano interactivo de mesas.

4. Alcance y propuesta de valor

4.1. Alcance

De acuerdo con el enunciado, en este parcial se implementan el **inicio de sesión** y el **formulario de reservas**. El cliente usa el formulario para reservar en línea

y el personal del restaurante lo usa para registrar reservas a nombre de un cliente (por ejemplo, las recibidas por teléfono). El diseño contempla tres roles: **cliente**, **empleado** y **administrador**. Como el administrador hereda todas las funciones del empleado, el rol de administrador que exige el enunciado queda cubierto, y el rol de empleado comparte sus mismas pantallas.

Además del formulario, en la primera fase se implementan la consulta de reservas del cliente, la cancelación con la política de reembolso y la agenda del día del personal. Los pagos se **simulan**: el sistema registra la modalidad de pago y emite el comprobante, pero no se conecta con una pasarela real. Las demás funciones (modificación de reservas, gestión de usuarios y mesas, estadísticas, reactivación de cuentas y promoción automática a VIP) quedan **diseñadas** en este documento para una fase posterior, y sus datos iniciales se cargan con el script SQL del Anexo **??**. Quedan fuera del alcance la pasarela de pagos real y la aplicación móvil nativa.

4.2. Propuesta de valor

La Tabla 1 resume los elementos que diferencian a Brindis de un sistema de reservas convencional.

Tabla 1
Elementos diferenciadores de Brindis

Elemento	**Descripción**	**Soporte en el diseño**
Reserva por ocasión	El cliente elige primero qué celebra (cumpleaños, aniversario, propuesta, cita romántica, negocios o casual). La ocasión sugiere la zona y filtra los planes de decoración que le corresponden.	Enumeración `Ocasion` y *Builder*
Plano interactivo con bloqueo temporal	El salón se muestra por zonas (VIP, general y terraza). Cada mesa aparece libre, reservada, bloqueada o seleccionada, y al elegirla queda bloqueada cinco minutos para que nadie más la tome.	Servicio de disponibilidad
Pedido anticipado y servicios extra	El cliente puede preordenar platos y contratar decoración o barra de cócteles; el total se recalcula al instante.	*Decorator*
Clientes VIP y *strikes*	Los clientes VIP tienen beneficios (cancelación más flexible y pago en el restaurante); las cancelaciones tardías y las inasistencias generan *strikes*.	*Factory Method* y *Proxy*
Restricciones alimentarias e indicaciones de servicio	Las alergias y lo que se debe preparar llegan como alerta a la agenda del personal.	*Decorator*

5. Análisis

El análisis sigue el enfoque orientado a objetos basado en casos de uso propuesto por Larman (2004) y la clasificación de requerimientos funcionales y no funcionales de Pressman y Maxim (2020).

5.1. Actores y roles

Tabla 2
Actores del sistema

Actor	Responsabilidades	¿Quién lo crea?
Cliente (general o VIP)	Inicia sesión, registra reservas, elige mesas en el plano, agrega pedido y servicios extra, define el pago, consulta el estado de sus reservas y las modifica o cancela.	Él mismo desde la aplicación, o un empleado.
Empleado	Consulta la agenda del día, registra, modifica o cancela reservas a nombre de los clientes, crea clientes que no están registrados y actualiza el estado de las reservas.	Un administrador.
Administrador	Hace todo lo que hace el empleado y, además, consulta estadísticas, crea empleados y administradores, asciende empleados a administrador, asciende clientes a VIP y activa o desactiva usuarios.	Otro administrador.

Nota. Elaboración propia.

5.2. Requerimientos funcionales

En la Tabla 3, la columna *Fase* indica si el requerimiento se implementa en este parcial (1) o si queda diseñado para una entrega posterior (2).

Tabla 3
Requerimientos funcionales

ID	Descripción	Prioridad	Fase
RF01	Iniciar sesión con correo y contraseña y redirigir al usuario a su panel según su rol.	Alta	1
RF02	Permitir que un cliente nuevo se registre por su cuenta; queda con rol cliente y tipo general.	Alta	1
RF03	Restringir cada función según el rol del usuario autenticado y la propiedad de la reserva.	Alta	1
RF04	Seleccionar la ocasión y el tipo de reserva (por mesas o restaurante completo).	Alta	1

ID	Descripción	Prioridad	Fase
RF05	Consultar la disponibilidad por fecha, hora, número de comensales y zona.	Alta	1
RF06	Mostrar un plano interactivo y permitir seleccionar una o varias mesas, que quedan bloqueadas temporalmente.	Alta	1
RF07	Agregar un pedido anticipado con productos de la carta.	Media	1
RF08	Agregar servicios extra (decoración y barra de cócteles) y recalcular el valor total.	Alta	1
RF09	Registrar restricciones alimentarias y observaciones.	Alta	1
RF10	Seleccionar la modalidad de pago (adelanto, pago completo o pago en el restaurante cuando aplique) y emitir el comprobante.	Alta	1
RF11	Confirmar la reserva, generar un código único y mostrar el resumen.	Alta	1
RF12	Permitir al personal registrar reservas a nombre de un cliente y crear al cliente si no existe.	Alta	1
RF13	Permitir al cliente consultar sus reservas y el estado de cada una.	Media	1
RF14	Cancelar una reserva aplicando la política de reembolso y de *strikes*, y emitir el comprobante del reembolso.	Media	1
RF15	Mostrar al personal la agenda del día con alertas, y permitirle cambiar el estado de las reservas.	Alta	1
RF16	Modificar una reserva aplicando los precios vigentes y cobrando la diferencia del adelanto.	Media	2
RF17	Enviar credenciales genéricas por correo a los clientes creados por el personal y exigir el cambio de contraseña en el primer ingreso.	Media	2
RF18	Permitir al administrador crear empleados y administradores, ascender usuarios y activarlos o desactivarlos.	Media	2
RF19	Mostrar al administrador estadísticas generales de reservas, ocupación e ingresos.	Media	2
RF20	Gestionar mesas, zonas, horarios de atención, carta y planes de servicios extra.	Media	2
RF21	Desactivar clientes por inactividad y permitir su reactivación con un *token* enviado por correo.	Baja	2
RF22	Promover automáticamente a VIP a los clientes que cumplan los criterios definidos.	Baja	2

Nota. Elaboración propia.

5.3. Requerimientos no funcionales

Tabla 4
Requerimientos no funcionales

ID	Atributo	Descripción
RNF01	Usabilidad	La reserva se completa en un máximo de cuatro pasos guiados y en menos de tres minutos para un usuario nuevo, siguiendo las heurísticas de Nielsen (1993).
RNF02	Seguridad	Las contraseñas se almacenan con *hash* BCrypt; cada operación pasa por un control de acceso por rol; las consultas usan `PreparedStatement` para evitar inyección SQL.
RNF03	Integridad	Una mesa no puede quedar en dos reservas activas con horarios cruzados; la verificación, el registro de la reserva y su comprobante se ejecutan en una misma transacción.
RNF04	Rendimiento	Las consultas de disponibilidad responden en menos de dos segundos con hasta 50 mesas.
RNF05	Mantenibilidad	Agregar un nuevo rol, un nuevo tipo de servicio extra o una nueva ocasión no exige modificar las clases existentes (principio abierto/cerrado).
RNF06	Trazabilidad	Todo movimiento de dinero genera un comprobante con el empleado que lo procesó, si lo hubo.
RNF07	Portabilidad	La interfaz es adaptable (*responsive*) y funciona en navegadores modernos de escritorio y móvil.

5.4. Reglas de negocio

Los valores numéricos de la Tabla 5 (tiempos, porcentajes y umbrales) son parámetros configurables del sistema.

Tabla 5
Reglas de negocio

ID	Tema	Regla
RN01	Duración	Cada reserva dura 120 minutos por defecto. Una mesa no admite dos reservas activas (vigente o cliente en mesa) con franjas que se crucen.
RN02	Capacidad	La suma de las capacidades de las mesas seleccionadas debe ser igual o mayor que el número de comensales.
RN03	Bloqueo temporal	Al seleccionar una mesa, esta queda bloqueada para esa franja durante cinco minutos. El bloqueo se libera al confirmar, al cancelar el proceso o al vencer el tiempo.
RN04	Restaurante completo	Una reserva de restaurante completo asocia automáticamente todas las mesas en servicio y exige que ninguna tenga reservas ni bloqueos en esa franja.

ID	Tema	Regla
RN05	Horario	La reserva debe quedar dentro del horario de atención y hacerse con al menos una hora de anticipación y como máximo con 60 días.
RN06	Modalidad de pago	*Adelanto*: el cliente paga un porcentaje del total (30 % por defecto). *Completo*: paga el 100 %. *En el restaurante*: paga el 0 % al reservar, y solo se permite si el cliente es VIP o si la reserva la registra el personal.
RN07	Precios y modificaciones	El precio de cada producto o servicio se guarda al agregarlo. Lo que se agrega en una modificación usa el precio vigente en ese momento, no el de la reserva original. Si el total aumenta, el cliente paga la diferencia necesaria para mantener el porcentaje del adelanto. El saldo se liquida el día de la reserva.
RN08	Cancelación de cliente general	Si cancela al menos cuatro horas antes, recibe el reembolso total de lo pagado. Si cancela después, se retiene el 10 % de lo pagado.
RN09	Cancelación de cliente VIP	Puede cancelar sin penalidad hasta una hora antes. Si cancela después, se retiene el 10 % de lo pagado y recibe un *strike*.
RN10	*Strikes*	La inasistencia y el comportamiento indebido generan un *strike*. Al tercer *strike*, un cliente VIP pasa a ser general, y un cliente general no puede ser promovido a VIP.
RN11	Promoción a VIP	Un cliente pasa a VIP si lo asciende un administrador o, de forma automática, si acumula 10 reservas completadas o un consumo de $2.000.000 sin tener tres *strikes*.
RN12	Creación de usuarios	El cliente se registra solo o lo crea un empleado; el empleado solo lo crea un administrador; el administrador solo lo crea otro administrador. Un cliente creado por el personal recibe por correo una contraseña genérica que debe cambiar en su primer ingreso.
RN13	Activación	Un cliente sin actividad durante 180 días queda inactivo y puede reactivarse solo con un *token* enviado a su correo. Un empleado o administrador desactivado solo puede ser reactivado por un administrador. Un administrador no puede desactivarse a sí mismo y siempre debe existir al menos un administrador activo.
RN14	Comprobantes	Cada movimiento de dinero (adelanto, liquidación, reembolso total o parcial) genera un comprobante.
RN15	Código	Cada reserva tiene un código alfanumérico único de ocho caracteres.

5.5. Diagrama de casos de uso

La Figura 1 muestra los casos de uso del módulo. El administrador es una especialización del empleado, por lo que hereda todos sus casos de uso. CU04 es el caso central: incluye siempre la selección de mesas y la definición del pago, y puede extenderse con el pedido anticipado y los servicios extra.

Figura 1
Diagrama de casos de uso de Brindis

IMAGEN: Diagrama de casos de uso. Actores: Cliente, Empleado y Administrador (hereda de Empleado). Casos CU01 a CU12; CU04 Registrar reserva incluye CU05 (seleccionar mesas) y CU06 (definir pago), y se extiende con CU07 (pedido anticipado) y CU08 (servicios extra).

Nota. Elaboración propia. La flecha con triángulo vacío entre actores indica que el administrador hereda los casos de uso del empleado.

5.6. Descripción de los casos de uso principales

Tabla 6
Especificación del caso de uso CU01 Iniciar sesión

Actores	Cliente, empleado, administrador.
Precondición	El usuario está registrado y activo.
Flujo básico	1. El usuario ingresa su correo y contraseña. 2. El sistema busca el usuario por correo y lo construye con la subclase de su rol. 3. El sistema compara la contraseña con el *hash* almacenado. 4. El sistema crea la sesión. 5. El sistema redirige al cliente a sus reservas y al personal a la agenda del día.
Flujos alternos	2a. El correo no existe o la contraseña no coincide: se muestra "Correo o contraseña incorrectos" sin indicar cuál dato falló. 2b. El usuario está inactivo: se informa y, si es cliente, se ofrece enviar el *token* de reactivación. 4a. La contraseña es genérica: se exige cambiarla antes de continuar.
Postcondición	Existe una sesión activa asociada al usuario y a su rol.

Tabla 7

Especificación del caso de uso CU04 Registrar reserva

Actores	Cliente (reserva en línea), empleado y administrador (reserva a nombre de un cliente).
Precondición	El usuario inició sesión.
Flujo básico	1. El usuario elige la ocasión, el tipo de reserva, la fecha, la hora y el número de comensales. 2. El sistema muestra el plano con el estado de cada mesa en esa franja y resalta la zona sugerida para la ocasión. 3. El usuario selecciona una o varias mesas (CU05) y el sistema las bloquea cinco minutos. 4. El usuario agrega, si lo desea, un pedido anticipado (CU07) y servicios extra (CU08), e indica restricciones alimentarias. 5. El sistema recalcula el total. 6. El usuario define la modalidad y el medio de pago (CU06). 7. El usuario confirma. 8. El sistema verifica los permisos, valida las reglas RN01 a RN06, guarda la reserva, emite el comprobante y muestra el código.
Flujos alternos	1a. Si el actor es del personal, primero busca al cliente por documento; si no existe, lo registra (CU11). 1b. Si elige restaurante completo, el sistema omite el paso 3 y asocia todas las mesas (RN04). 3a. La capacidad seleccionada no alcanza: el sistema lo indica y no permite continuar. 8a. El bloqueo venció y otro usuario tomó la mesa: el sistema informa y regresa al paso 2. 8b. Se eligió pago en el restaurante sin cumplir RN06: el sistema lo rechaza.
Postcondición	La reserva queda en estado *vigente*, con su código, sus mesas, su detalle de pago y su comprobante.

Nota. Elaboración propia.

Tabla 8

Especificación del caso de uso CU09 Modificar o cancelar reserva (cancelación)

Actores	Cliente (sus propias reservas), empleado y administrador.
Precondición	La reserva está vigente.
Flujo básico	1. El usuario elige la reserva y la opción cancelar. 2. El sistema calcula las horas que faltan y, según el tipo de cliente, el valor a reembolsar (RN08, RN09). 3. El sistema muestra el reembolso y la retención. 4. El usuario confirma. 5. El sistema cambia el estado a *cancelada*, libera las mesas, emite el comprobante de reembolso total o parcial y, si aplica, registra un *strike*.
Flujos alternos	1a. Un cliente intenta cancelar una reserva ajena: el control de acceso lo rechaza. 2a. No hubo pagos: la reserva se cancela sin comprobante.
Postcondición	La reserva queda cancelada y el reembolso queda registrado.

Nota. Elaboración propia.

Tabla 9
Especificación del caso de uso CU10 Gestionar agenda del día

Actores	Empleado, administrador.
Precondición	El usuario inició sesión con un rol del personal.
Flujo básico	1. El usuario abre la agenda. 2. El sistema muestra las reservas del día ordenadas por hora, con el cliente (y si es VIP), las mesas, la ocasión, las alertas de restricciones y las indicaciones de servicio. 3. El usuario cambia el estado de una reserva: *cliente en mesa* al llegar, *completada* al liquidar el saldo o *inasistida*. 4. El sistema guarda el cambio; si es una inasistencia, registra un *strike*; si es una liquidación, emite el comprobante.
Flujos alternos	1a. El usuario elige otra fecha. 3a. La transición no es válida (por ejemplo, completar una reserva cancelada): el sistema la rechaza.
Postcondición	El estado de la reserva queda actualizado.

Nota. Elaboración propia.

5.7. Ciclo de vida de una reserva

La Figura 2 resume los estados de una reserva y los eventos que producen cada transición.

Figura 2
Diagrama de estados de la reserva

IMAGEN: Diagrama de estados de la reserva: VIGENTE pasa a CLIENTE_EN_MESA y luego a COMPLETADA; desde VIGENTE también puede pasar a CANCELADA o a INASISTIDA; la modificación se hace sobre una reserva VIGENTE.

Nota. Elaboración propia.

6. Diseño

6.1. Arquitectura MVC por capas

Brindis sigue el patrón arquitectónico Modelo–Vista–Controlador, en el que la vista presenta la información, el controlador atiende las peticiones del usuario y el modelo contiene la lógica de negocio y los datos (Fowler, 2002). Para separar mejor las responsabilidades, el modelo se divide a su vez en tres capas internas: servicios y patrones, acceso a datos y entidades. La Figura 3 muestra la organización resultante.

Figura 3
Arquitectura MVC por capas de Brindis

IMAGEN: Arquitectura MVC por capas: Vista (plantillas HTML y plano.js), Controlador (AuthController, ReservaController, PersonalController), Modelo de servicios y patrones (ReservaServiceProxy, ReservaServiceImpl, UsuarioFactory, ReservaBuilder, ReservaDecorator, DisponibilidadService, PagoService), capa DAO, entidades y base de datos MySQL.

Nota. Elaboración propia. Los controladores no acceden directamente a la lógica de reservas: siempre pasan por `ReservaServiceProxy`, que verifica los permisos.

Las responsabilidades de cada capa son:

- **Vista.** Plantillas Thymeleaf para el inicio de sesión, el registro, el asistente de reserva en cuatro pasos, las reservas del cliente y la agenda del personal. El archivo `plano.js` dibuja el salón y consulta la disponibilidad sin recargar la página.
- **Controlador.** Recibe las peticiones, convierte los datos del formulario en objetos de transferencia (DTO) y los entrega a la capa de servicios. No contiene lógica de negocio.
- **Modelo: servicios y patrones.** Contiene la lógica de negocio y las clases de los cuatro patrones: `UsuarioFactory` (*Factory Method*), `ReservaBuilder` (*Builder*), `ReservaServiceProxy` (*Proxy*) y `ReservaDecorator` (*Decorator*).
- **Modelo: acceso a datos.** Interfaces DAO y sus implementaciones con JDBC, que aíslan el SQL del resto del sistema (Alur et al., 2003).
- **Modelo: entidades.** Clases que representan los conceptos del dominio y DTO que transportan datos entre capas.

6.2. Diagrama de clases del modelo

La Figura 4 presenta las clases de entidad. `Usuario` es abstracta y se especializa en `Cliente` y `Empleado`; `Administrador` hereda de `Empleado`, lo que expresa que el administrador puede hacer todo lo que hace un empleado. La reserva pertenece a un cliente, puede haber sido creada por un empleado, ocupa una o varias mesas y se compone de su pedido anticipado y sus servicios extra.

Figura 4
Diagrama de clases del modelo (entidades)

IMAGEN: Diagrama de clases del modelo: Usuario (abstracta) con Cliente y Empleado; Administrador hereda de Empleado. Reserva se asocia con Cliente, Empleado (creador) y Mesa, y se compone de DetallePedido, ReservaServicioExtra y DetallePago; tiene varios ComprobantePago. PlanServicioExtra (abstracta) con PlanDecoracion y PlanBarraCocteles.

Nota. Elaboración propia. Se omiten los métodos *get* y *set*. El rombo relleno indica composición. `HorarioAtencion` y `TokenReactivacion` se omiten por espacio; aparecen en el modelo relacional.

La enumeración `Ocasion` sostiene la reserva por ocasión sin necesidad de un patrón adicional: cada valor conoce la zona que se sugiere y el motivo de decoración que se ofrece.

Tabla 10
Valores de la enumeración Ocasion

Ocasión	Zona sugerida	Decoración ofrecida	Indicación para el personal
CUMPLEANOS	General	Cumpleaños	Preparar la canción al servir el postre.
ANIVERSARIO	VIP	Aniversario	Luz tenue y música suave.
PROPUESTA	VIP	Propuesta	Coordinar con el cliente el momento de la propuesta.
ROMANTICA	Terraza	Romántica	Mesa apartada y servicio discreto.
NEGOCIOS	VIP	Ninguna	Servicio ágil y sin interrupciones.
CASUAL	Cualquiera	Ninguna	Servicio estándar.

6.3. Modelo relacional de la base de datos

El modelo relacional se obtuvo a partir del modelo de clases, aplicando la normalización hasta la tercera forma normal (Elmasri y Navathe, 2016). Las tres subclases de usuario se almacenan en una sola tabla con la columna `rol`; las relaciones de muchos a muchos entre reservas y mesas, productos o servicios se resuelven con las tablas `reserva_mesa`, `reserva_detalle_pedido` y `reserva_servicio_extra`, que además conservan el precio vigente al momento de agregar cada elemento (RN07).

El estado de la mesa se trata en dos niveles. En la tabla `mesa` se guarda solo su estado físico (`DISPONIBLE` o `FUERA_DE_SERVICIO`). Los estados *reservada* y *bloqueada temporalmente* dependen de la franja consultada, por lo que se calculan a partir de las tablas `reserva_mesa` y `bloqueo_mesa`. Así, una misma mesa puede estar reservada a las 8 p. m. y libre a la 1 p. m.

La Figura 5 presenta las trece tablas y sus relaciones; el script completo de creación se encuentra en el Anexo **??**.

Figura 5
Modelo relacional de la base de datos `brindis_db`

IMAGEN: Modelo relacional de brindis_db con sus trece tablas (usuario, token_reactivacion, mesa, bloqueo_mesa, horario_atencion, reserva, reserva_mesa, item_menu, reserva_detalle_pedido, plan_servicio_extra, reserva_servicio_extra, detalle_pago y comprobante_pago), con llaves primarias, foráneas y cardinalidades.

Nota. Elaboración propia. Los tipos de dato completos, las restricciones `CHECK` y los índices se detallan en el Anexo **??**.

Las principales restricciones de integridad son:

- `usuario.correo`, (`usuario.tipo_documento`, `usuario.documento`) y `reserva.codigo` son únicos.
- `reserva.id_creador` admite nulos: es nulo cuando el cliente reservó desde la aplicación. Lo mismo ocurre con `comprobante_pago.id_empleado`.
- `detalle_pago.porcentaje` está entre 0 y 100; vale 100 si la modalidad es `COMPLETO` y 0 si es `EN_RESTAURANTE`.
- `mesa.capacidad` está entre 1 y 20, y `reserva.num_comensales` es mayor que cero.
- Los índices sobre `reserva_mesa.id_mesa` y `reserva.fecha_hora_inicio` aceleran la verificación de cruces de horario (RN01).

6.4. Diagrama de clases DAO

El patrón DAO encapsula el acceso a la fuente de datos detrás de interfaces, de modo que la lógica de negocio no depende de SQL ni del motor de base de datos (Alur et al., 2003). Brindis define una interfaz genérica `GenericDAO<T, ID>` con las operaciones CRUD; cada agregado del dominio tiene una interfaz que la extiende con consultas propias y una implementación JDBC. Todas las implementaciones obtienen sus conexiones de `ConexionBD`, que administra un *pool* de conexiones. `UsuarioDAOImpl` usa la fábrica de usuarios para reconstruir cada registro con la subclase de su rol.

Figura 6
Diagrama de clases de la capa de acceso a datos (DAO)

IMAGEN: Diagrama de clases DAO: interfaz GenericDAO¡T, ID¿ extendida por UsuarioDAO, MesaDAO, ReservaDAO, CatalogoDAO y PagoDAO; cada una con su implementación JDBC, que usa ConexionBD.

Nota. Elaboración propia. Línea continua con triángulo: herencia de interfaces. Línea discontinua con triángulo: implementación. Línea discontinua con flecha: dependencia.

6.5. Patrones de diseño aplicados

Se aplican exactamente cuatro patrones del catálogo de Gamma et al. (1994): dos creacionales y dos estructurales. La Tabla 11 resume el problema que resuelve cada uno; las subsecciones siguientes presentan su estructura y su implementación.

Tabla 11
Patrones de diseño aplicados en Brindis

Tipo	Patrón	Clases principales	Problema que resuelve
Creacional	*Factory Method*	`UsuarioFactory` `ClienteFactory` `EmpleadoFactory` `AdministradorFactory`	Cada rol se crea con datos y reglas distintas (tipo de cliente, *strikes*, credenciales genéricas). Centralizar la creación evita condicionales repartidos por el código y permite agregar roles sin modificar lo existente.
Creacional	*Builder*	`ReservaBuilder`	La reserva asocia cliente, mesas, consumos, servicios extra y pago, con datos obligatorios y opcionales. Un constructor con más de diez parámetros sería ilegible, y la reserva solo debe existir si cumple las reglas de negocio.
Estructural	*Proxy*	`ReservaServiceProxy` `UsuarioServiceProxy`	Las mismas operaciones se permiten o se niegan según el rol del usuario y la propiedad de la reserva. El control de acceso no debe mezclarse con la lógica de negocio.
Estructural	*Decorator*	`ReservaDecorator` `DecoracionDecorator` `BarraCoctelesDecorator`	Los servicios extra se combinan libremente y cada uno cambia el costo y las indicaciones de servicio. Crear una subclase por cada combinación sería inviable.

6.5.1. Patrón creacional Factory Method: creación de usuarios por rol

Factory Method define una interfaz para crear un objeto, pero deja que las subclases decidan qué clase instanciar (Gamma et al., 1994). En Brindis, la clase abstracta `UsuarioFactory` contiene el método `registrar()`, que ejecuta los pasos comunes a todos los roles (validar que el correo y el documento no existan, cifrar la contraseña y guardar), y delega en el método fábrica `crearUsuario()` la instanciación del objeto concreto. Cada subclase decide qué crear:

- `ClienteFactory` crea un `Cliente` general con cero *strikes*. Si el cliente lo registra un empleado, genera una contraseña genérica y la envía por correo con la obligación de cambiarla (RN12).
- `EmpleadoFactory` crea un `Empleado`.
- `AdministradorFactory` crea un `Administrador`.

`UsuarioService` guarda las fábricas en un mapa indexado por rol y elige la que corresponde; `UsuarioDAOImpl` usa el mismo mapa para reconstruir cada registro leído de la base de datos con su subclase, de modo que al iniciar sesión el sistema trabaja con el objeto del tipo correcto. Agregar un rol nuevo, por ejemplo "Cajero", solo requiere una nueva subclase y su fábrica (RNF05). Qué usuario puede crear a qué otro no lo decide la fábrica, sino el *Proxy* de usuarios.

Figura 7
Estructura del patrón Factory Method en Brindis

IMAGEN: Patrón Factory Method: clase abstracta UsuarioFactory con el método crearUsuario(); subclases ClienteFactory, EmpleadoFactory y AdministradorFactory, que crean Cliente, Empleado y Administrador. UsuarioService elige la fábrica según el rol.

6.5.2. Patrón creacional Builder: construcción validada de la reserva

Builder separa la construcción de un objeto complejo de su representación, de modo que el mismo proceso pueda crearlo paso a paso (Gamma et al., 1994). `ReservaBuilder` ofrece un método encadenable para cada parte de la reserva: cliente, creador, tipo, mesas, fecha, comensales, ocasión, productos del pedido, servicios extra, restricciones y pago. Todas las validaciones se concentran en `build()`: si falta un dato obligatorio o se viola una regla de negocio, se lanza una excepción y la reserva nunca llega a existir en un estado inválido. El constructor de la clase `Reserva` tiene visibilidad de paquete, de modo que `ReservaBuilder` es la única vía para crear una reserva. `ReservaServiceImpl` cumple el papel de director: decide en qué orden y con qué datos se invocan los métodos del *builder*.

Figura 8
Estructura del patrón Builder en Brindis

IMAGEN: Patrón Builder: ReservaServiceImpl (director) usa ReservaBuilder, con sus métodos encadenables y build(), para crear el producto Reserva.

6.5.3. Patrón estructural Proxy: control de acceso por rol

Proxy proporciona un sustituto de otro objeto para controlar el acceso a él (Gamma et al., 1994). Brindis usa un *proxy de protección*: `ReservaServiceProxy` implementa la misma interfaz `IReservaService` que el servicio real, `ReservaServiceImpl`, y los controladores solo conocen la interfaz. Antes de delegar cada llamada, el *proxy* consulta el usuario de la sesión y verifica:

- que un cliente solo reserve, consulte, modifique o cancele reservas a su propio nombre;
- que solo el personal (empleados y administradores) vea la agenda del día, registre reservas a nombre de otros y cambie estados a *cliente en mesa*, *completada* o *inasistida*;
- que solo un administrador consulte las estadísticas.

Como `Administrador` hereda de `Empleado`, la verificación `instanceof Empleado` concede al administrador todos los permisos del personal sin código adicional. Con la misma estructura, `UsuarioServiceProxy` protege la gestión de usuarios: un empleado solo puede crear clientes, solo un administrador crea empleados o administradores, un administrador no puede desactivarse a sí mismo y no se puede desactivar al último administrador activo (RN12, RN13). Separar estos controles de la lógica de negocio permite cambiar las políticas de acceso sin tocar `ReservaServiceImpl`.

Figura 9
Estructura del patrón Proxy en Brindis

IMAGEN: Patrón Proxy: ReservaController usa la interfaz IReservaService, implementada por ReservaServiceProxy (verifica el rol con SesionUsuario) y por ReservaServiceImpl (sujeto real), a la que el proxy delega.

6.5.4. Patrón estructural Decorator: costo de los servicios extra

Decorator agrega responsabilidades a un objeto de forma dinámica, envolviéndolo en otros objetos que comparten su misma interfaz (Gamma et al., 1994). La reserva

base calcula el costo del pedido anticipado; luego se envuelve con un decorador por cada servicio extra contratado. Cada decorador suma su precio, agrega su nombre a la descripción y añade sus propias indicaciones para el personal:

- `DecoracionDecorator` suma la tarifa del plan de decoración, que depende del motivo y del nivel (básico, medio o elaborado), y pide montar la decoración 20 minutos antes de la llegada.
- `BarraCoctelesDecorator` suma la tarifa del plan de barra (básico, carta completa o *premium show*); si el plan se cobra por persona, multiplica el precio por el número de comensales.

Como todos implementan `ComponenteReserva`, el servicio calcula el total sin importar cuántos servicios haya ni en qué orden se agregaron, y agregar un nuevo tipo de servicio (por ejemplo, música en vivo) solo requiere un nuevo decorador.

Figura 10
Estructura del patrón Decorator en Brindis

IMAGEN: Patrón Decorator: interfaz ComponenteReserva implementada por ReservaBase y por la clase abstracta ReservaDecorator, que envuelve otro componente; subclases DecoracionDecorator y BarraCoctelesDecorator. Ejemplo: 180.000 + 80.000 + 4 x 45.000 = 440.000.

6.6. Diagrama de clases de control

La Figura 11 integra los cuatro patrones con los controladores y la capa DAO. Los controladores dependen de las interfaces de servicio, que en tiempo de ejecución están implementadas por los *proxies*; `ReservaServiceImpl` dirige al *builder* y compone los decoradores; `UsuarioService` elige la fábrica de usuarios según el rol; y solo los servicios acceden a los DAO.

Figura 11
Diagrama de clases de control con los patrones aplicados

IMAGEN: Diagrama de clases de control: AuthController, ReservaController y PersonalController usan IUsuarioService e IReservaService; los proxies delegan en UsuarioService y ReservaServiceImpl; estos usan UsuarioFactory, ReservaBuilder, ComponenteReserva, DisponibilidadService y PagoService, que acceden a la capa DAO.

Nota. Elaboración propia. Azul claro: controladores e interfaces. Vino: *proxy*. Las flechas discontinuas son dependencias de uso; las de triángulo vacío, implementación.

6.7. Diagrama de secuencia: confirmar una reserva

La Figura 12 muestra cómo colaboran las clases cuando el cliente confirma una reserva. El controlador hace una sola llamada; el *proxy* verifica el permiso antes de delegar; y el servicio real usa el *builder* y los decoradores dentro de una misma transacción.

Figura 12
Diagrama de secuencia del caso de uso CU04 (confirmación)

IMAGEN: Diagrama de secuencia al confirmar una reserva: Cliente, ReservaController, ReservaServiceProxy (verifica permisos), ReservaServiceImpl, DisponibilidadService, ReservaBuilder, ComponenteReserva, ReservaDAO y PagoService (emite el comprobante del abono).

Nota. Elaboración propia. Las flechas discontinuas son respuestas. La franja sobre la línea de vida de `ReservaServiceImpl` indica el periodo de la transacción.

6.8. Diseño de la interfaz de usuario

La interfaz se diseñó para que la reserva se complete en cuatro pasos visibles en un indicador de progreso (ocasión, mesas, pedido y extras, y pago), con lenguaje cercano al usuario, colores consistentes para el estado de las mesas y mensajes que explican cómo corregir cada error, en línea con las heurísticas de Nielsen (1993). La paleta usa el color vino como color de marca; en el plano, el verde indica mesas libres, el rojo reservadas, el gris bloqueadas por otro usuario y el dorado las mesas seleccionadas. Las Figuras 13 a 16 presentan los prototipos de baja fidelidad.

Figura 13
Prototipos de inicio de sesión y paso 1 (ocasión)

IMAGEN: a) Inicio de sesión con correo, contraseña y enlace de registro. b) Paso 1 del asistente: tarjetas de ocasión (cumpleaños, aniversario, propuesta, casual), tipo de reserva (por mesas o restaurante completo), fecha, hora, comensales y zona sugerida.

Nota. Elaboración propia.

Figura 14
Prototipos del paso 2 (plano de mesas) y paso 3 (pedido y servicios extra)

IMAGEN: a) Paso 2: plano del salón por zonas (general, VIP y terraza) con mesas libres, reservadas, bloqueadas y seleccionadas, indicador de capacidad y tiempo de bloqueo. b) Paso 3: pedido anticipado, servicios extra sugeridos para la ocasión, restricciones alimentarias y resumen con el total.

Nota. Elaboración propia. Los precios están en pesos colombianos y son ilustrativos.

Figura 15
Prototipos del paso 4 (pago) y de las reservas del cliente

IMAGEN: a) Paso 4: modalidad de pago (adelanto, completo o en el restaurante, deshabilitada para clientes generales), medio de pago y política de cancelación. b) Listado de reservas del cliente con su estado y ventana de cancelación con el cálculo del reembolso.

Nota. Elaboración propia.

Figura 16
Prototipo de la agenda del personal

IMAGEN: Agenda del día del personal: menú lateral, tabla con hora, código, cliente (VIP), mesas, ocasión, alertas e indicaciones de servicio y estado de cada reserva, botón de reserva para cliente y alerta de restricciones alimentarias.

Nota. Elaboración propia. Las opciones de usuarios y estadísticas solo aparecen para el administrador.

7. Implementación

8. Casos de prueba

latex

Tabla 12
Casos de prueba funcionales

ID	Escenario	Resultado esperado	¿Cumple?
CP01	Inicio de sesión con credenciales válidas de cliente.	Redirige a “Mis reservas”.	

ID	Escenario	Resultado esperado	¿Cumple?
CP02	Inicio de sesión con contraseña incorrecta.	Mensaje genérico de error.	
CP03	Registro de un cliente nuevo desde la aplicación.	Queda con rol cliente, tipo general y cero *strikes* (*Factory Method*).	
CP04	Un cliente intenta abrir la agenda del personal.	Acceso denegado (*Proxy*).	
CP05	Un cliente intenta cancelar la reserva de otro cliente.	Acceso denegado (*Proxy*).	
CP06	Dos usuarios seleccionan la misma mesa en la misma franja.	El segundo la ve bloqueada y no puede tomarla (RN03).	
CP07	Reservar para 6 personas con mesas que suman 4 puestos.	Se rechaza por RN02 (*Builder*).	
CP08	Cliente general elige pago en el restaurante.	Se rechaza por RN06 (*Builder*).	
CP09	Pedido de $180.000, decoración de $80.000 y barra de $45.000 por persona para 4 personas.	Total de $440.000 (*Decorator*).	
CP10	Cliente general cancela 2 horas antes, con $132.000 pagados.	Reembolso de $118.800 y comprobante de reembolso parcial (RN08).	
CP11	Cliente VIP cancela 30 minutos antes.	Retención del 10 % y un *strike* (RN09).	
CP12	El personal marca una inasistencia a un cliente VIP con dos *strikes*.	El cliente pasa a general (RN10).	
CP13	Un empleado registra una reserva para un cliente nuevo.	Se crea el cliente con contraseña genérica y la reserva queda con el empleado como creador.	
CP14	Reserva de restaurante completo en una franja libre.	Se asocian todas las mesas en servicio (RN04).	

9. Video explicativo

10. Conclusiones

Los cuatro patrones aplicados responden a problemas reales del dominio y no se incluyeron de forma decorativa. *Factory Method* centralizó la creación de clientes, empleados y administradores, cada uno con sus propias reglas, y permite agregar roles sin modificar el código existente. *Builder* concentró en un solo lugar las reglas de negocio de la reserva (capacidad, horario, modalidad de pago), de modo que

ningún objeto `Reserva` puede existir en un estado inválido.

En el plano estructural, *Proxy* separó el control de acceso de la lógica de negocio: las mismas operaciones se permiten o se niegan según el rol y la titularidad de la reserva, y la herencia entre `Administrador` y `Empleado` hizo que los permisos del personal se extendieran al administrador sin código adicional. *Decorator* resolvió la combinación libre de servicios extra, cuyo costo e indicaciones de servicio se calculan sin importar cuántos se agreguen.

La arquitectura MVC por capas y el patrón DAO hicieron que cada cambio afecte a una sola capa. Distinguir entre el estado físico de la mesa y su estado en una franja, junto con el bloqueo temporal, evita las reservas duplicadas entre el canal en línea y el telefónico.

Finalmente, la reserva por ocasión, el plano interactivo y las alertas de restricciones alimentarias acercan el software a la operación real de un restaurante. Las reglas de cancelación y de *strikes*, por su parte, protegen al restaurante del trabajo que pierde cuando un cliente cancela tarde o no se presenta.

Referencias

Alur, D., Crupi, J., & Malks, D. (2003). *Core J2EE patterns: Best practices and design strategies* (2.ª ed.). Prentice Hall.

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Freeman, E., & Robson, E. (2020). *Head First design patterns* (2.ª ed.). O'Reilly Media.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Larman, C. (2004). *Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development* (3.ª ed.). Prentice Hall.

Nielsen, J. (1993). *Usability engineering*. Academic Press.

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.ª ed.). McGraw-Hill Education.